

LIG-TEAM CHATBOT PROJECT ("ASKYOURPDF")

Academic Project Report for University Graduation ("Soutenance")

□ Administrative Details

- **Project Title:** AskYourPdf: A High-Performance, Premium Workspace Interface for AI-Powered Retrieval-Augmented Generation (RAG)
 - **Academic Year:** 2025/2026
 - **University:** *[Insert University Name]*
 - **Faculty / Department:** *[Insert Department, e.g., Computer Science / Software Engineering]*
 - **Degree Program:** *[Insert Degree, e.g., Bachelor of Science / Master in Computer Science]*
 - **Lead Software Architect (Frontend, Security, & Backend Server Systems):** Mohamed Islem
 - **Backend AI & Document Pipeline Engineer:** Ahmed (GitHub: @0ahmed0ghoul)
 - **Academic Supervisor:** *[Insert Supervisor's Name and Title]*
-

□ Table of Contents

1. Executive Summary
2. Introduction & Project Context
 - Problem Statement
 - Objectives
 - Collaboration Scope & Detailed Division of Labor
3. Dynamic Premium Design System (Frontend)
 - The 10-Theme Color Engine
 - The 7-Font Global Typography Engine
 - Interactive Micro-Animations & 3D WebGL Features
4. Backend System Engineering & Database Schema
 - FastAPI Server Framework
 - MongoDB Integration & Document Collections
 - OAuth2 & JWT Security Infrastructure
 - File Management & I/O Controllers

- *Community Boards & Social Features*
5. **AI core: Document Ingestion, Vectors & RAG (Retrieval-Augmented Generation)**
 - *Multi-Format Text Extraction & Parsing*
 - *Vector Database Indexing (FAISS)*
 - *Contextual Prompt Synthesis & LLM Integrations*
 6. **Functional Architecture & Core Workspace Features**
 - *Premium Chat Workspace*
 - *Interactive Document Preview & Citation Panel*
 - *Drag-and-Drop Session Management*
 7. **State Management & Network Synchronization**
 - *Redux Toolkit Global Client State*
 - *RTK Query Network Layer & Automatic Token Refresh*
 8. **Directory Map & Project Structure**
 9. **System Workflow & Architecture Diagrams**
 10. **Setup, Installation & Deployment Guide**
 11. **Conclusion & Future Outlook**
-

1. Executive Summary

The **AskYourPdf (LIG-Team Chatbot)** is a highly specialized, premium workspace environment designed to revolutionize the way users interact with complex documents. By bridging high-performance **Retrieval-Augmented Generation (RAG)** backend algorithms with an ultra-premium, interactive React-based frontend, the platform transforms static files (PDF, DOCX, TXT) into smart conversational nodes.

This report outlines the **full-stack software engineering, database integration, security systems, and UI/UX design** implemented for the project. Built utilizing a modern stack composed of **React 19, Vite, Redux Toolkit, Tailwind CSS v4, FastAPI (Python), and MongoDB**, the application is optimized for visual excellence, micro-interactions, and secure state management. Key accomplishments include a dynamic **10-theme color engine**, a **7-font global typography switcher**, an advanced user authentication platform supporting secure **HttpOnly cookie tokens**, persistent multi-session chat workspaces, and a premium 3D particle simulation field using **Three.js (React Three Fiber)**.

2. Introduction & Project Context

Problem Statement

In academic, legal, and corporate environments, extracting specific information from extensive

documents (such as textbooks, contracts, and research papers) is notoriously time-consuming. Classic keyword searches fail to capture context, while general-purpose Large Language Models (LLMs) suffer from hallucinations and lack knowledge of private files.

Objectives

To address these issues, the **LIG-Team Chatbot** provides an isolated, intelligence-scoped workspace that:

1. Allows users to securely upload private documents up to 10MB.
2. Generates localized vector embeddings to allow semantic, context-aware querying.
3. Synthesizes precise answers sourced strictly from the uploaded material (RAG).
4. Engages the user in an ultra-modern, beautiful workspace that maintains visual appeal, high readability, and fluid transitions to maximize productivity.

Collaboration Scope & Detailed Division of Labor

The engineering distribution for this project was clearly structured. **Mohamed Islem** spearheaded the complete application design, the entire frontend workspace, client-side security flows, and the bulk of the FastAPI backend server infrastructure (including databases, security schemas, user sessions, cookie transactions, and CRUD systems). **Ahmed** engineered the core backend text ingestion pipeline, vector indices, and natural language question-and-answer search.

- **Mohamed Islem (Lead Software Architect):**
 - **Frontend UI/UX Architecture:** Designed the entire client workspace, layout grids, settings panel, dynamic theme engine (10 themes), and global typography selectors (7 fonts).
 - **Interactive Visuals:** Engineered the 3D particle WebGL field using Three.js / React Three Fiber (`Antigravity.jsx`), GSAP ScrollTrigger transitions, smooth inertia scrolling with Lenis, and custom cursor click sparks.
 - **Backend Server Infrastructure:** Designed and programmed the main FastAPI application (`server.py`), logging setups, and middleware CORS configurations.
 - **Database Engineering (MongoDB):** Developed the entire MongoDB connector schema, integrating active dynamic databases to model user credentials, chat histories, active chat rooms, and public discussion comments.
 - **Security & User Authentication:** Developed the end-to-end security framework including bcrypt password hashing, JWT signature controls (`src/auth.py`), HTTP-Only session cookies parsing/issuance middleware, token refresh handlers (`/refresh`), and user registration, login, and logout endpoints.
 - **State & Networking Orchestration:** Standardized client-side state hooks via Redux Toolkit and managed parallel server state queries through RTK Query, implementing automatic re-authentication interceptors.
 - **Client/Server CRUD Systems:** Programmed controllers for file retrieval APIs, chat session list generation, starring features, and clean document purging routines.

- **Ahmed (Backend AI Core Developer):**
 - **Document Processing Pipeline:** Structured standard file extraction for DOCX, TXT, and PDF documents using libraries like PyMuPDF (fitz), PDFMiner, and pytesseract OCR parsing.
 - **Vector DB & Index Ingestion (ingest.py, pipeline.py):** Configured sentence transformer bindings (all-MiniLM-L6-v2) to map text chunks into high-density 384-dimensional mathematical arrays and integrated local FAISS indexes.
 - **Q&A Generative Logic (chatbot.py):** Programmed search similarity algorithms and wired LLM completion APIs to formulate answers derived from context.
-

3. Dynamic Premium Design System (Frontend)

One of the defining aspects of the AskYourPdf frontend is its commitment to visual elegance. Instead of creating a static dashboard, the workspace is treated as an active, customizable canvas designed to match the user's focus, ambient lighting, or device hardware.

□ The 10-Theme Color Engine

The system implements a premium CSS Custom Property override system, enabling users to swap the entire layout palette with a single click.

Theme Identifier	Name	Description	Key Accents
indigo <i>(Default)</i>	Midnight Void	Original deep cosmic dark mode layout	Cosmic Indigo (#4f46e5), Charcoal Panel (#13141A)
emerald	Abyssal Slate	Deep slate ocean dark mode with cyan	Electric Cyan (#06b6d4), Slate Ocean (#0f172a)
rose	Volcanic Ash	Warm stone dark mode with deep magma	Magma Crimson (#e11d48), Stone Grey (#292524)
amber	Cyberpunk Gold	Radiant yellow-gold layout with dark amber	Glowing Amber (#f59e0b), Warm Cyberpunk (#211a0a)
dracula	Dracula Purple	Classic developer aesthetic with neon pink	Neon Dracula Pink (#ff79c6), Dracula Slate (#44475a)
forest	Deep Forest	Lush pine green with glowing lime accents	Forest Emerald (#10b981), Moss Green (#064e3b)
monochrome	Pure OLED Black	Ultra-high contrast pitch black for OLED	Pure White (ffffff), Ink Black (000000)

Theme Identifier	Name	Description	Key Accents
cherry	Cherry Blossom	Dark maroon with soft spring pink accents	Sakura Pink (#ec4899), Velvet Maroon (#4a1224)
synthwave	Synthwave Neon	Retro-futuristic 80s violet and neon cyan	Cyber Neon Cyan (#06b6d4), Retro Violet (#312e81)
sepia	Vintage Espresso	Extremely comfortable warm coffee dark mode	Dark Espresso (#3d342b), Warm Sepia Gold (#d97706)
light	Clean Snow	Pristine white layout with indigo accents	Crisp Indigo (#4f46e5), Snow White (ffffff)

The 7-Font Global Typography Engine

To enhance readability and provide personalization, a typography engine dynamically injects font family declarations across all parent DOM trees:

1. **Outfit (Default):** Clean, modern, and extremely friendly geometric sans-serif.
2. **Inter:** The absolute industry standard of crisp, high-legibility UI design.
3. **Plus Jakarta Sans:** A wide, expansive, premium corporate look.
4. **Space Grotesk:** A tech-forward, futuristic developer-oriented geometric face.
5. **DM Sans:** Highly minimal, lightweight, and modern aesthetic.
6. **JetBrains Mono:** Tech/hacker aesthetic providing monospace focus.
7. **Playfair Display:** High-end luxury serif for a sophisticated, editorial vibe.

□ Interactive Micro-Animations & 3D WebGL Features

- **Three.js / React Three Fiber (Antigravity.jsx):** Renders a high-performance 3D particle field under the landing page. Particles are affected by mouse magnetism, shifting coordinates dynamically in real-time. If the mouse is idle, an automatic sinusoidal mathematical wave animates the vector particles.
- **GSAP & ScrollTrigger:** Implements scroll-driven layout reveals. Landing page panels glide, rotate, and scale based on active scrolling indices.
- **Lenis Smooth Scrolling:** Eliminates browser-native jumpy scrolling, wrapping inputs in a beautiful physical inertia engine.
- **Custom UI Sprites:**
 - **ClickSpark:** Emit geometric light bursts at click coordinates.
 - **LightRays:** Multi-angle dynamic light beams forming a futuristic background

halo.

- LetterGlitch: Retro cyberpunk terminal text scramble interactions.
 - RotatingText & ScrollFloat: Liquid-smooth text transitions.
-

4. Backend System Engineering & Database Schema

Engineered and Developed by Mohamed Islem

The backend core of the application utilizes **FastAPI (Python)**, chosen for its asynchronous capability, automated API documentation (Swagger), and extreme processing speed. The server is structured around clean REST endpoints communicating with a local **MongoDB** database cluster to orchestrate system configurations.

MongoDB Integration & Document Collections

The database holds persistent workspace files, structured around four primary collections:

1. **users:** Models user credentials and profiles.

```
{
  "_id": "ObjectId",
  "username": "string",
  "email": "string",
  "hashed_password": "string (bcrypt)",
  "role": "string"
}
```

2. **chat_sessions:** Tracks active workspace nodes mapping back to a specific document.

```
{
  "_id": "ObjectId",
  "chat_id": "string (uuid4)",
  "username": "string",
  "filename": "string",
  "is_starred": "boolean",
  "created_at": "datetime"
}
```

3. **chats:** Stores chronological text threads.

```
{
  "_id": "ObjectId",
  "chat_id": "string",
  "username": "string",
  "question": "string",
  "answer": "string",
  "sources": [{"source": "string", "similarity": "float"}],
  "timestamp": "datetime"
}
```

4. **comments:** Tracks posts published on the public discussion board.

```
{
  "_id": "ObjectId",
  "username": "string",
  "text": "string",
}
```

```
"timestamp": "datetime"  
}
```

□ OAuth2 & JWT Security Infrastructure

Session safety is constructed utilizing cryptographic signatures and strict cookie restrictions:

- **Password Hashing:** Passwords are cryptographically salted and hashed using **bcrypt** via the `passlib` library before DB insertion.
- **Double-Token Generation:** Valid logins generate both a short-term **Access Token** (expires in 15 minutes) and a long-term **Refresh Token** (expires in 7 days) signed via **HMAC-SHA256 (JWT)**.
- **HttpOnly Cookies Transmission:** To mitigate Cross-Site Scripting (XSS) risks, tokens are never exposed to browser javascript. Instead, they are returned in HTTP headers via `Set - Cookie` labeled as `httponly=True`, `samesite="lax"`, and `path="/"`.
- **Guard Middleware:** Custom Dependency injections (`get_current_user`) intercept active routers, extract cookie parameters, decode JWT headers, validate username signatures, and raise standard 401 exceptions if credentials are missing or expired.

□ File Management & I/O Controllers

To avoid server disk clutter, file management modules utilize logical database constraints:

- **Limits Validation:** Prior to accepting file uploads, the server queries the database count of active workspaces, enforcing a maximum limit of 10 file upload nodes per user.
- **I/O Storage Scoping:** Document uploads are partitioned cleanly by username on the storage disk (`data/user_files/{username}/{filename}`).
- **CRUD Deletion Integrity:** Triggering a chat deletion safely drops the corresponding session mapping, flushes the chronological message history, removes the physical file from the storage directory, and rebuilds the localized vector database indexes.
- **Binary Delivery Interfaces:** Dynamic endpoint APIs (`GET /user-files/{filename}/view`) read raw PDF documents directly into stream outputs, converting raw texts or DOCX layouts into optimized inline HTML wrappers.

□ Community Boards & Social Features

To foster cooperation, a global **Comments Section** allows active users to share notes:

- Users can fetch (`GET /comments`) or publish (`POST /comments`) message packets.
- Security guards ensure users can only delete comments matching their active username (`DELETE /comments/{comment_id}`).

5. AI Core: Document Ingestion, Vectors & RAG

Engineered and Developed by Ahmed

The AI processing segment focuses on turning static, text-heavy documents into searchable data nodes by generating local vector embeddings and applying semantic retrieval strategies.

□ Multi-Format Text Extraction & Parsing

- Handles diverse formats including **PDFs**, **DOCX**, and **TXT** files.
- Uses **PyMuPDF (fitz)** and **PDFMiner** to extract raw text blocks while maintaining structural readability.
- Integrates **Tesseract OCR** capabilities via the **pytesseract** library, allowing the parser to extract and read embedded text inside images and scanned pages within documents.

□ Vector Database Indexing (FAISS)

- **Text Chunking:** Extracted texts are split into readable chunks using logical character offsets to maintain local semantic context.
- **Mathematical Encoding:** Chunks are translated into high-dimensional vector spaces using the **all-MiniLM-L6-v2** model from the **sentence-transformers** library, which maps texts into 384-dimensional mathematical arrays.
- **FAISS Storage Index:** Indexes are stored in secure local partitions (`data/user_vector_db/{username}`). Chunks are indexed inside **FAISS** (**Facebook AI Similarity Search**) matrices.

□ Contextual Prompt Synthesis & LLM Integrations

- **Semantic Matching:** When a user queries the bot, the query is mapped to the same 384-dimensional vector space. Chunks showing high cosine similarity are fetched from the local FAISS index.
- **Context-Injected Prompts:** Top matching text chunks are formatted alongside prompt templates, instructing the generative model to answer strictly using the provided context.
- **LLM API Integration:** Prompts are sent to generative models (e.g. OpenAI completions) to produce accurate, context-aware answers, return citation filenames, and compute similarity scores.

6. Functional Architecture & Core Workspace Features

```
graph TD
    User([User Client]) -->|Interacts & Logs In| UI[React Frontend Workspace]
    UI -->|Cookie Credentials / JWT| RTK[Redux RTK Query]
    RTK -->|JWT Credentials / HttpOnly| BE[FastAPI Backend - Port 8000]

    subgraph Frontend Components [Developed by Mohamed Islem]
        UI --> Home[Home / Landing Page]
        UI --> AuthForms[Login, Register & Logout Routes]
        UI --> ChatWorkspace[Chat / Active Workspace]
        ChatWorkspace --> DocPreview[Live Document Preview]
        ChatWorkspace --> ChatWindow[Active Chat Thread]
        ChatWorkspace --> Customizer[Theme & Font Customizer]
    end
    end
```



```

subgraph Backend Routing & DB [Developed by Mohamed Islem]
  BE --> AuthRouter[Auth, Cookie & Refresh Middleware]
  BE --> CRUD[Sessions, Stars, Files, & Forums CRUD APIs]
  CRUD --> Mongo[(MongoDB Server Database)]
end

subgraph Backend AI RAG Core [Developed by Ahmed]
  BE --> IngestPipeline[Doc Parser, Chunking & OCR Ingestion]
  BE --> QueryRouter[Retrieval-Augmented Generator Q&A Core]

  IngestPipeline -->|384 Dimensions| FAISS[(FAISS Vector Storage)]
  QueryRouter -->|Cosine Semantic Search| FAISS
  QueryRouter -->|Augmented Prompt Context| LLM[LLM Generation Engine]
end

```

1. Premium Chat Workspace (Chat . jsx)

The core workspace is divided into an ergonomic multi-panel grid system:

- **Left Control Panel:** Handles session lists (showing uploaded documents). Starred chats are grouped at the top, and current document storage constraints are animated with slot-tracking bars (e.g., maximum 10 documents limit).
- **Workspace Settings:** Houses the real-time Theme and Font switchers, plus database update controllers.
- **Document Hub / Steppers:** Houses quick-start visual guides and embedded video tutorials with custom-built media player controls.

2. Interactive Document Preview & Citation Panel

To prevent workspace fragmentation, an inline visualizer utilizes an `iframe` sandbox to view documents natively on the left side of the screen. Users can pop out the viewer to a new tab or reference active text side-by-side with chat inputs. When responses are output, precise details (similarity percentages and source filenames) appear in custom citation nodes.

3. Drag-and-Drop Session Management (Chatbot . jsx)

Uploading a document is fully intuitive:

- Entering drag states overlays an active backdrop glow.
- Dropping a valid document immediately communicates with the backend ingestion endpoint (`/chat-sessions`).
- The document is parsed, indexed, and a new persistent session is added to the sidebar.

7. State Management & Network Synchronization

Engineered and Developed by Mohamed Islem

To keep complex client panels synced with backend database endpoints, the application implements a unified state management layer on the client side.

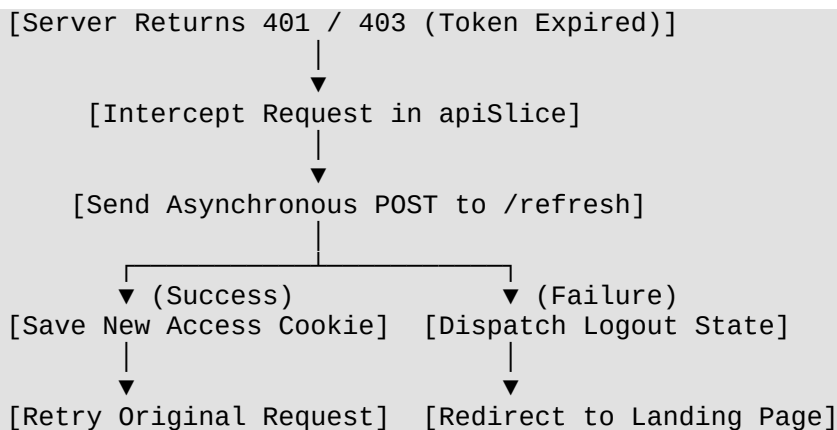
Redux Toolkit Global Client State

- **User Session Sync (`authSlice.jsx`):** Parses user details directly from cookies. Reducers manage authentication states, updating cookie records upon successful registration or login, and purging them on logout.
- **Layout & Workspace States:** Coordinates global settings, sidebar states, active chat references, and custom typography/theme selections.

□ RTK Query Network Layer & Automatic Token Refresh

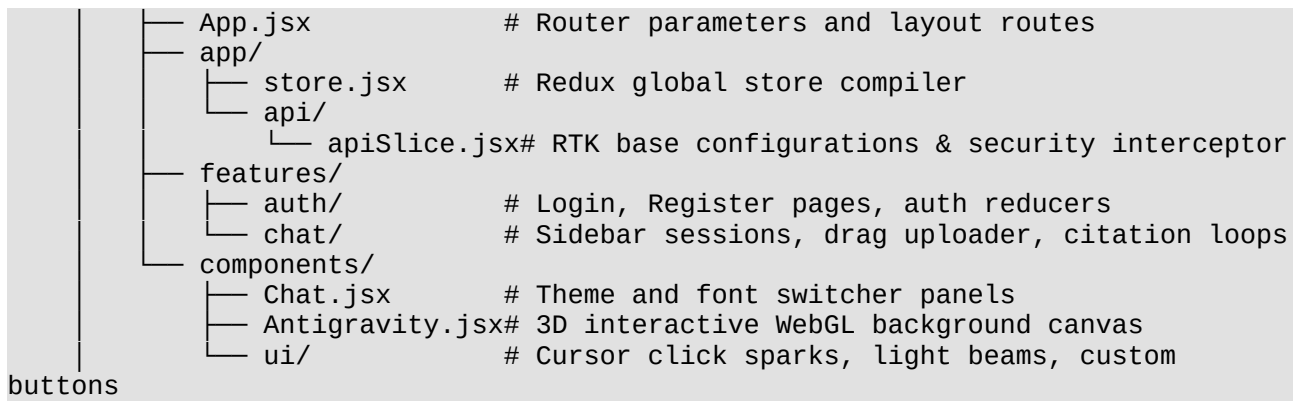
Network synchronization uses **RTK Query** to manage caching, request states, and server endpoints:

- **Base API Injection:** `apiSlice.jsx` injects endpoints across the workspace (e.g., `/auth`, `/register`, `/chat-sessions`, `/comments`), avoiding repetitive backend query configurations.
- **Asynchronous Token Interceptor:** Coordinates the re-authentication sequence:



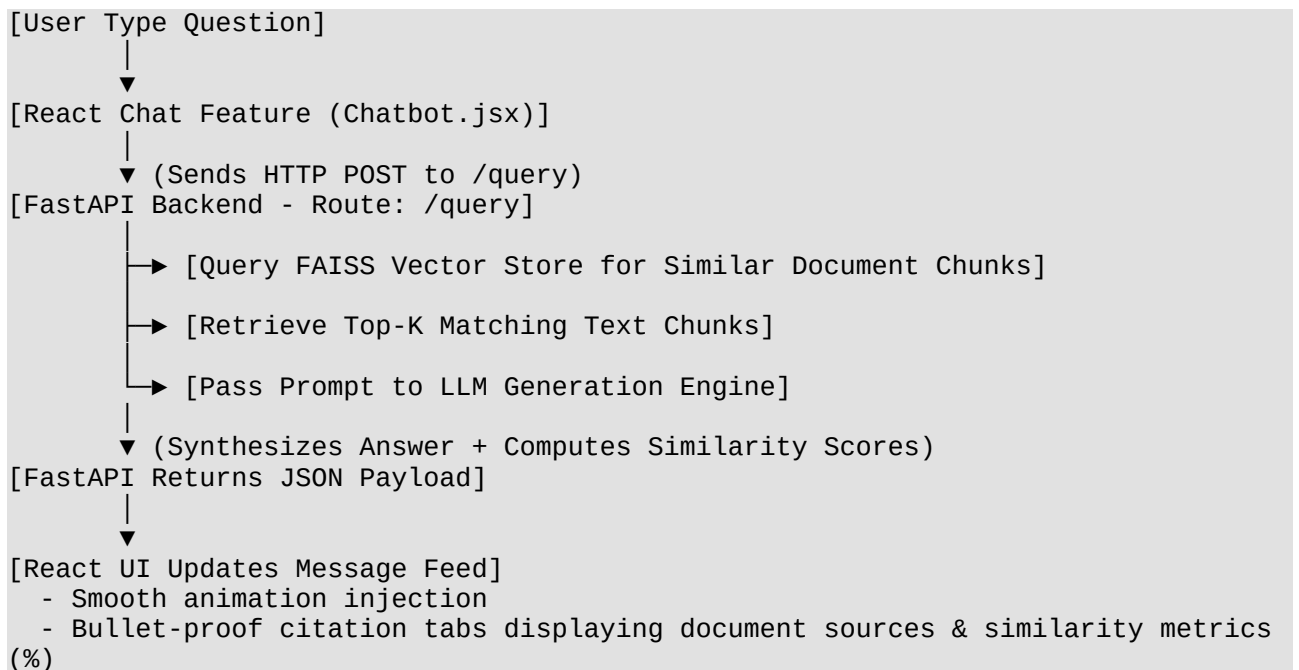
8. Directory Map & Project Structure

LIG-Team-project/	
├── B/	# BACKEND SYSTEM (FastAPI & MongoDB)
│ ├── server.py	# Main API server routes & core endpoints
(Mohamed Islem)	
│ ├── requirements.txt	# Backend dependencies (FastAPI, FAISS, PyMuPDF)
│ └── src/	
│ ├── auth.py	# Password hashing, JWT & Security (Mohamed Islem)
Islem)	
│ ├── config.py	# Environment settings (Mohamed Islem)
(Ahmed)	
│ ├── chatbot.py	# Prompt configurations & QA answering logic
│ ├── ingest.py	# Embedding execution & FAISS indexing (Ahmed)
│ ├── pipeline.py	# Text extraction, PDF OCR parsing (Ahmed)
│ ├── retriever.py	# Similarity calculation matches (Ahmed)
│ ├── search.py	# Dense/lexical matching parameters (Ahmed)
│ ├── utils.py	# Parsing utilities and helpers
│ └── data/	# Storage node for files, vector bin indexes, &
DB	
└── frontend/	# FRONTEND APPLICATION (React 19 & Tailwind v4)
│ ├── package.json	# Frontend dependencies (R3F, Redux, GSAP)
│ └── src/	
│ └── main.jsx	# App entry point



9. System Workflow & Architecture Diagrams

During active sessions, client interactions execute clean transactional pipelines:



10. Setup, Installation & Deployment Guide

Prerequisites

- **Python:** version 3.10.0 or higher.
- **MongoDB:** Local community server active on `mongodb://127.0.0.1:27017`.
- **Node.js & npm:** Node version 18.0.0+ / npm version 9.0.0+.

1. Backend Ingestion Server Setup

1. Navigate to the backend directory B:

```
cd B
```

2. Establish and launch a python virtual environment:

```
python -m venv venv  
venv\Scripts\activate
```

3. Install dependencies:

```
pip install -r requirements.txt
```

4. Run the FastAPI local server:

```
uvicorn server.py --reload --port 8000
```

2. Frontend Application Setup

1. Navigate to the frontend directory:

```
cd ../frontend
```

2. Install packages:

```
npm install
```

3. Launch the hot-reloading development server:

```
npm run dev
```

11. Conclusion & Future Outlook

The **AskYourPdf Workspace** delivers a high-performance, beautifully interactive, and context-aware workspace. Through the deployment of modern design elements (React 19, Redux, Tailwind v4, Three.js) and secure backend integrations (FastAPI, MongoDB, OAuth2, FAISS RAG), the project demonstrates that complex technical tools can be made highly accessible, transparent, and aesthetically pleasing.

Key Milestones Achieved:

1. **Lead Architecture Coordination:** Successful deployment of fully modular full-stack client-server structures.
2. **Robust Backend Integrity:** Secure implementation of MongoDB collections, session cookies validation middleware, security handshakes, and public social comment threads.
3. **Dynamic Customization Canvas:** Customizers supporting 10 color settings and 7 typographic options to boost productivity.
4. **Premium WebGL Aesthetics:** Smooth 3D particle canvas renders improving user interaction.

Full-Stack Architecture, Interactive UI/UX, Security Systems, & Database Integrations engineered by Mohamed Islem. Document Processing Pipeline, FAISS Vector Indexing, & AI Retrieval Logic developed by Ahmed.

